IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

Welcome to IEM 4723 Information Systems Design

- **Course Codes:** IEM 4723 (Undergraduate) / IEM 5723 (Graduate)
- **Semester:** Fall 2026
- **Classroom:** EN310
- **Lecture Time:** Tuesdays and Thursdays, 10:30am-11:45am
- **Instructor:** Dr. Paritosh Ramanan
- **Office:** 354 Engineering North
- **Email:** paritosh.ramanan@okstate.edu

Course Description - Part 1

- Design and implementation of information systems
- Git and GitHub for version control and coursework management
- Data storage using relational (RDBMS) and key-value databases
- Hands-on implementation and manipulation of databases
- Containerization with Docker for portable, reproducible applications

Course Description - Part 2

- Data visualization using Python and Streamlit
- Building interactive dashboards and web applications
- AI agent development with open-source frameworks
- LLM integrations and MCP (Model Context Protocol)
- Enterprise tools (ERP and GIS)

Course Topics (Tentative)

- **Introduction and Git Basics (Lec 1)** - Version control fundamentals
- **Git Basics Continued (Lec 2)** - Workflow and collaboration
- **Introduction to Containers (Lecs 3-6)** - Docker fundamentals
- **Entity-Relationship Modeling (Lec 7)** - Database design
- **Relational Database Systems (Lecs 8-13)** - RDBMS and SQL
- **Midterm Exam** - covering material through Lecture 13
- **NoSQL and Key-Value Storage (Lecs 14-15)** - Redis
- **Data Visualization with Streamlit (Lecs 16-20)** - App development
- **AI Computing Ecosystem (Lecs 21-24)** - LLMs and AI Agents
- **Enterprise Tools (Lec 25)** - ERP and GIS
- **Project Presentations (Lecs 27-28)**

Textbooks and Resources

- **[H]** Harrington, J.L., *Relational Database Design and Implementation*, 4th Edition
- **[D]** Iliev, B., *Introduction to Docker* (free eBook)
- **[S]** Streamlit Documentation and *30 Days of Streamlit* (free)
- **[R]** DigitalOcean, *How To Manage a Redis Database* (free eBook)
- **References:**
  - Valacich et al., *Essentials of Systems Analysis and Design*
  - Whitten & Bentley, *Systems Analysis and Design Methods*
  - Class Notes/handouts (Git and GitHub basics, AI Agent Fundamentals)

Prerequisites

- IEM 1412 or equivalent programming experience
- Basic understanding of computer systems
- Willingness to learn new technologies and tools

Technology Requirements

Students will need access to a computer for daily work, assignments, and projects. Various free and open-source tools will be used throughout the course.

Grading Policy

Assignment	Weight
Assignment	15%
Midterm Exam	25%
Project	35%
Final Exam	25%

- Two exams will be given in this course
- Exams are closed book and closed notes

Letter Grade Scale (subject to curving)

- **A:** 90% and above
- **B:** 80% - 89.9%
- **C:** 70% - 79.9%
- **D:** 60% - 69.9%
- **F:** Below 60%

Class Policy

- Attendance is essential for success in this course
- Excessive absences may impact your final grade
- Students are responsible for all material covered in class
- Regular participation in discussions is expected
- Exams must be taken on scheduled dates
- Documentation required for make-up exams

Communication

All course communication will be through Canvas and official OSU email.

Student Support Resources

- Free resources available for financial, mental health, and suicide prevention support
- Visit: https://ssc.okstate.edu/
- Counseling and psychological services available
- Academic success resources through the university
- Disability services available for students with documented needs

Don't Hesitate to Seek Help

Whether it's academic or personal, the university has resources available to support your success and well-being.

Office Hours and Contact

- **Office:** 354 Engineering North
- **Email:** paritosh.ramanan@okstate.edu
- **Office Hours:** By appointment (send email to schedule)
- **Response Time:** I will strive to respond to emails within 24-48 hours
- **Canvas Inbox:** Use the Canvas messaging system for course-related questions

Getting Help

- Attend office hours or schedule an appointment
- Use Canvas Inbox for course questions
- Check announcements and discussions first
- Be specific in your questions

Git: Distributed Version Control System

- All repositories are equivalent and contain the entire history
- Commits are local (don't need to be connected to a central repository)
- But a central repository is still useful
- Bare repositories don't have working directories

Installing Git

Windows:

- Download from: https://git-scm.com/download/win
- Run installer with default settings

macOS:

- Option 1: Install Xcode Command Line Tools
- `git --version` (prompts to install if needed)
- Option 2: Use Homebrew
- `brew install git`

Linux (Ubuntu/Debian):

- `sudo apt update`
- `sudo apt install git`

Configuring Git

After installing Git, configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email your@email.com
```

Set your default editor:

```
git config --global core.editor vim
```

Verify configuration:

```
git config --list
```

GitHub GUI Options

GitHub Website:

- Web-based interface for managing repositories
- Create, fork, clone repositories
- Manage issues, pull requests
- View file history and branches

GitHub Desktop:

- Free desktop app for Windows and macOS
- Visual interface for common Git operations
- Easy branching and merging
- Automatic sync with GitHub
- Download from: https://desktop.github.com/

IDE Integration:

- VS Code, Jetbrains IDEs have built-in Git support
- Visual commit history and branch management

Creating a Git Repository

```
cd project # change to project directory
git init # create empty repo
git add filename # add file to index
git commit # commit the file to the repo
```

Git: Four Locations for Your Files

- **Working copy:** Your actual files on disk
- **Staging area (index):** Files prepared for the next commit
- **Local repository:** Your local history of commits
- **Remote repository:** Repository on a server (e.g., GitHub)

Git Cheat Sheet (Without Branches)

```
working<--> index <--> local <--> remote

git clone # clone a git repository
git init # initialize git for the current directory
git status # status of all files
git log # history of commits
git diff # show changes between working and local copies
git ls-files # what files are being managed by git?
git add <file> # stage a file
git reset HEAD <file> # unstage a file
git checkout -- <file> # revert a file
git commit # commit files
git fetch # fetch from remote repo
git merge # merge local repo files
git pull # fetch and merge changes
git push # push from local to remote repo
```

Using git log and git show

```
# which commits is a file in?
git log <file>

# when was a file added?
git log --diff-filter=A -- <file>

# see changes in a commit
git show <commit>

# list files that changed in a commit
git show --name-only <commit>

# see version of file at a given commit
git show <commit>:<file>
```

Using git diff

- `git diff` - Show differences between working directory and index
- `git diff --cached` - Show differences between index and most recent commit
- `git diff HEAD` - Show differences between working directory and most recent commit
- `git diff <commit> <file>` - Compare workspace file with file in previous commit
- `git diff <commit1> <commit2> <file>` - Compare file from two different commits

Specifying Revisions

- Use the hash of the commit
- `HEAD` - most recent commit
- `HEAD~` - parent of most recent commit
- `HEAD~~` or `HEAD~2` - grandparent
- `^2` - which parent (if merge commit)

Bare Repositories

- Repositories that are shared
- Typical use: shared repository acts as the master copy
- No working directory, therefore "bare"
- Directory names end in `.git` by convention

Bare Repo and Normal Repo (Fresh Project)

In the directory where you want the bare repo stored:

```
git init --bare project.git
```

Then in the directory where you want your normal repo:

```
git clone /path_or_url/to/project.git
```

Push for the first time:

```
git push origin master
```

Bare Repo for Existing Git Project

In the directory where you want the bare repo:

```
git clone --bare /path_or_url/to/existing_repo
```

Then in the directory of the existing repo:

```
git remote add origin /path_or_url/to/bare_repo
```

Using Git the First Time

Set configuration variables:

```
git config --global user.name "Your Name"
git config --global user.email your@email.com
git config --global core.editor vim
```

Code Management: The Problem

[scale=0.7, transform shape] [rectangle, draw, fill=blue!10, text width=3cm, align=center] (d1) Dev 1
app.py v1; [rectangle, draw, fill=blue!10, text width=3cm, align=center, right=2cm of d1] (d2) Dev 2
app.py v1;
[rectangle, draw, fill=red!10, text width=3cm, align=center, below=1.5cm of d1] (sf) Shared Folder
/projects;
[-¿, thick] (d1) – node[right] Submit (sf); [-¿, thick] (d2) – node[left] Submit (sf); [-¿, thick, dashed] (sf) – node[left] Download (d1); [-¿, thick, dashed] (sf) – node[right] Download (d2);

Problems:

- No change history
- Files get overwritten

Git Solution: Distributed VCS

[scale=0.7, transform shape] [rectangle, draw, fill=blue!10, text width=2.5cm, align=center] (r1) Local Repo; [rectangle, draw, fill=blue!10, text width=2.5cm, align=center, right=2.5cm of r1] (r2) Local Repo; [rectangle, draw, fill=orange!15, text width=2.5cm, align=center, below=2cm of r1] (gh) GitHub; [-¿, thick] (r1) – node[midway, left] push/pull (gh); [-¿, thick] (r2) – node[midway, right] push/pull (gh);

Benefits:

- History everywhere
- Safe offline work
- Easy branching
- Remote backup

What is a fork?

- A fork is a clone of a repo, but...
- The owner of the original repo can see the files in your fork

Big Picture: Course Repository Workflow

Course repo
`disyslab/IEM4723`
(upstream)

↑ pull updates

Your local repo
`jinx:IEM4723`

↓ push commits

Your fork
`jdoe1/IEM4723`
(origin)

Getting Updates from Course Repo

First set the upstream repo (only needed once):

```
git remote add upstream \
git@github.com:disyslab/IEM4723.git
```

Then pull from the upstream repo:

```
git pull upstream master
```

Finally, push to keep your fork updated:

```
git push
```

Getting Updates from Your Fork

- Simple: `git pull`
- However, we do not recommend changing files in your fork using GitHub
- This creates extra synchronization work